CS1312
Algorithms Design and Analysis

Sorting

Heap and Heap Sort

Heap

- Heap data structure is used for:
  - Heap sort
  - Priority queue

- The (binary) heap data structure is an array object that we can view as a nearly complete binary tree.
  - A completely filled on all levels except possibly the lowest.

Heap

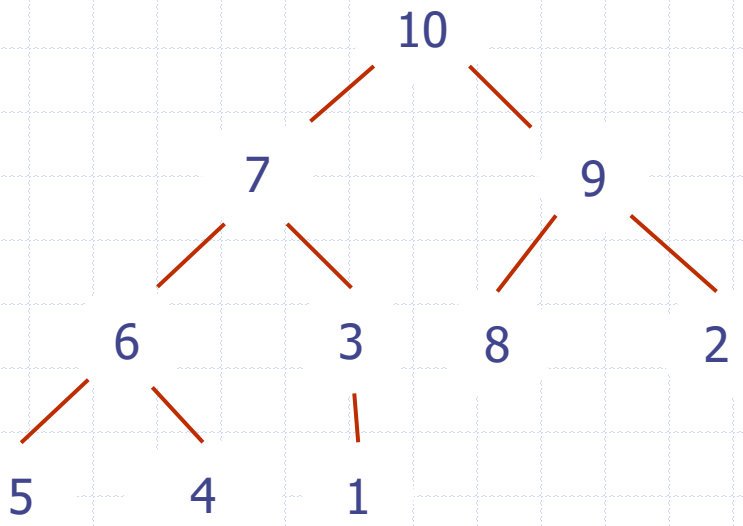
- Each node of the tree corresponds to an element of the array.
- An array **A** that represents a heap is an object with two attributes:
  - A.length: number of elements in the array.
  - A.heap-size: represents how many elements in the heap are stored within array **A**.
- Although A[1..A.length] may contain numbers, only the elements in A[1..A.heap-size], where
  - 0 <= A.heap-size <= A.length, are valid elements of the heap.

Heap (cont.)

- Example: the array **A** could be represented in the following Max-heap:

index	1	2	3	4	5	6	7	8	9	10
	10	7	9	6	3	8	2	5	4	1

- The root of the tree in A[1].



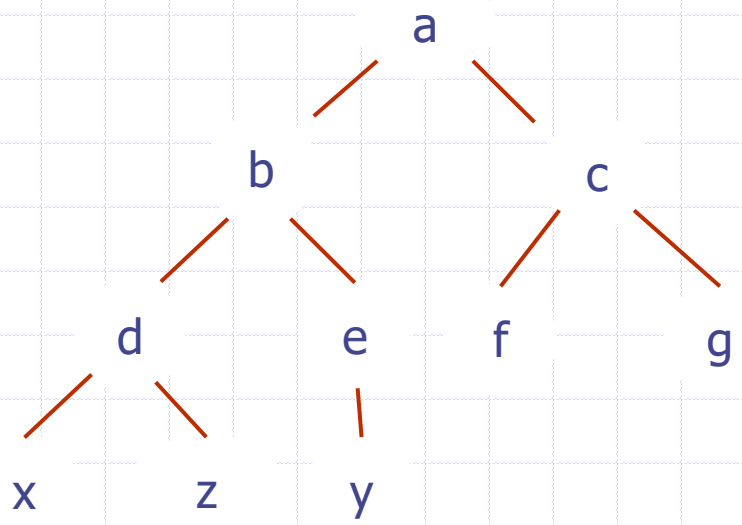

```
PARENT(i)
1  return ⌊i/2⌋

LEFT(i)
1  return 2i

RIGHT(i)
1  return 2i + 1
```

Example

1	2	3	4	5	6	7	8	9	10
a	**b**	**c**	**d**	**e**	**f**	**g**	**x**	**z**	**y**



Heap (Cont.)

- There are two kinds of heaps: max heap and min heap.
- In both kinds, the value in the nodes satisfy the heap property.
  - Parents have a higher priority than any of their children.

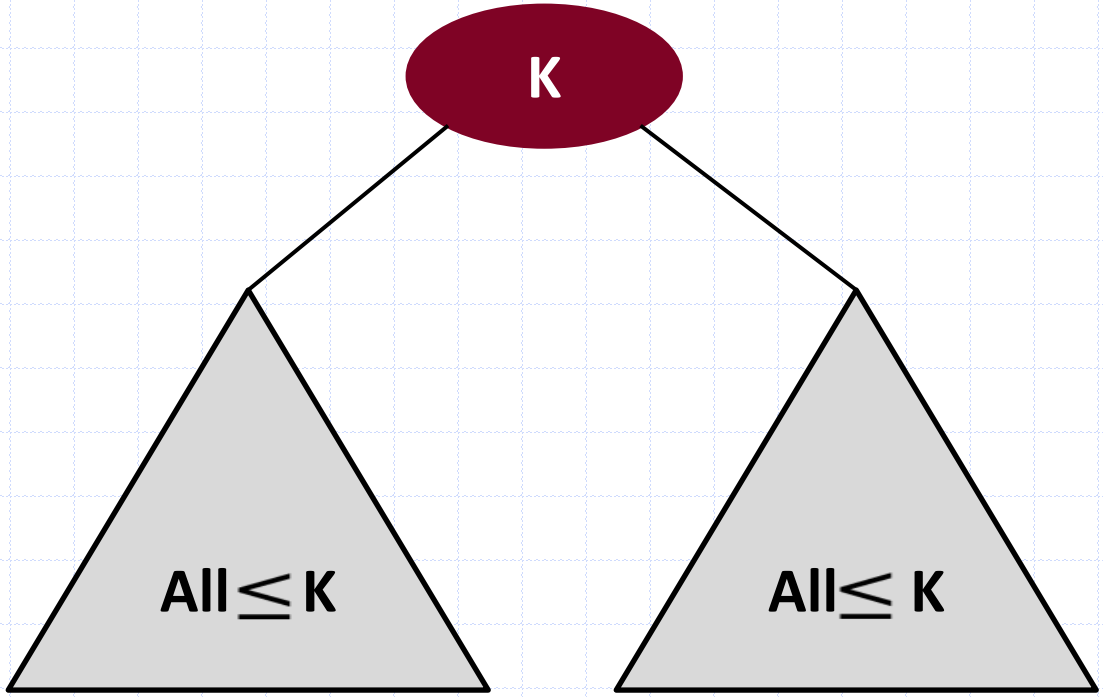
Max Heap

- Satisfies the max heap property that is for every node *i* other than the root,

$$A[parent(i)] >= A[i]$$

- The **largest** element in the max heap is stored in the root.

Max Heap



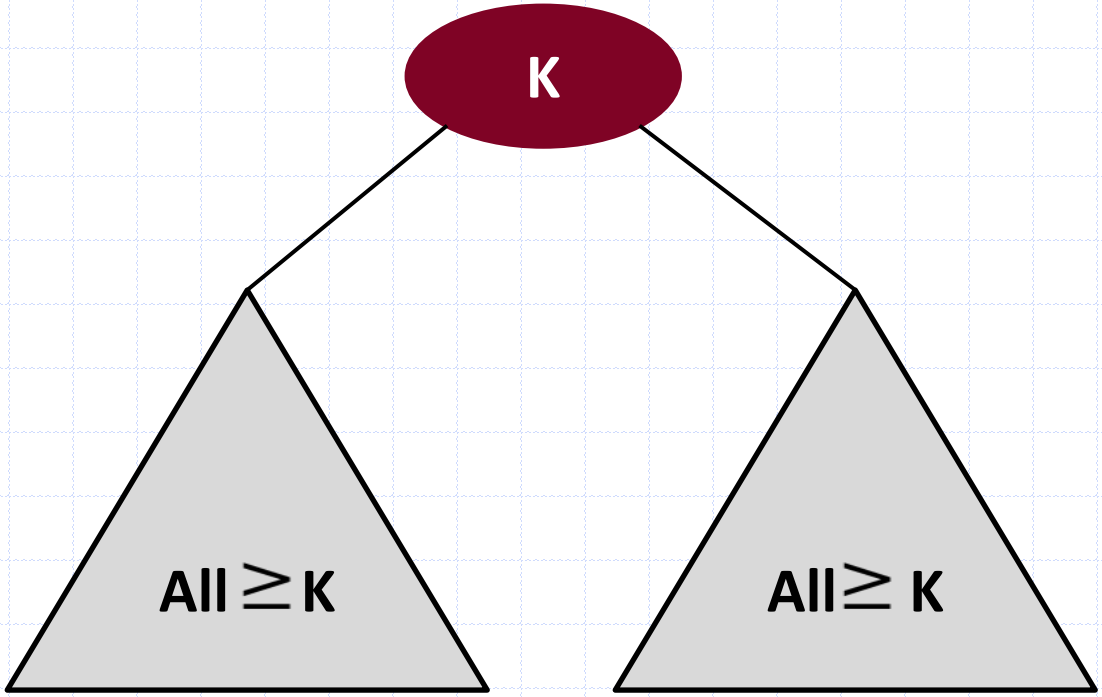
Min Heap

- Satisfies the min-heap property is that for every node *i* other than the root:

$$A[\text{Parent}(i)] \leq A[i]$$

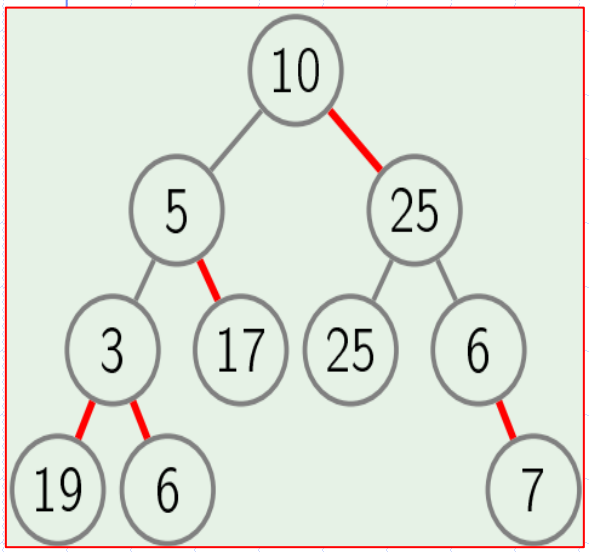
- The **smallest** element in a min-heap is at the root.
- Commonly used to implement **priority queues**.

Min Heap

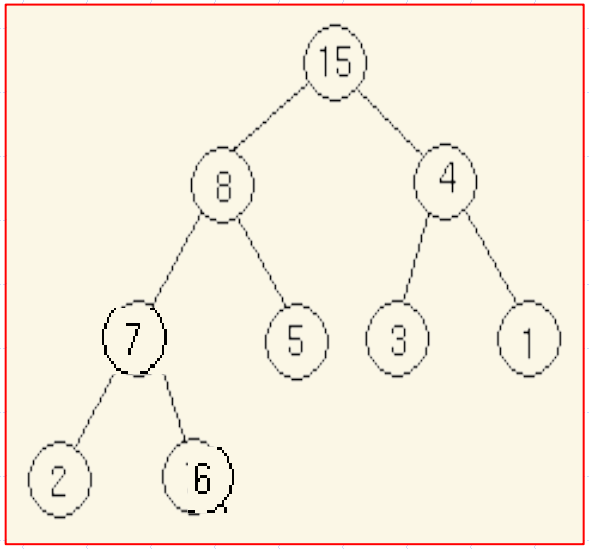


Example

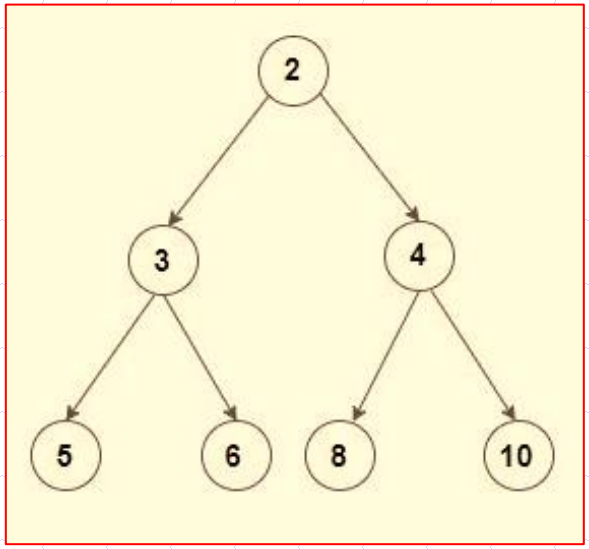
Not a heap



Max heap



Min heap



Heap

- The **height of a node** in a heap is the number of edges on the longest simple downward path from the node to a leaf.
- The **height of the heap** is the height of its root.
- Since a heap of *n* elements is based on a complete binary tree, its **height** is $\theta(\log n)$
- The basic operations on heaps run in time **at most** proportional to the height of the tree and thus take $O(\log n)$.

Heap (Cont.)

Some Basic Procedures :

1. The **MAX-HEAPIFY** procedure: is the key to maintaining the max-heap property.
   - Runs in *O(log n).*
2. The **BUILD-MAX-HEAP** procedure: produces a max-heap from an unordered input array.
   - Runs in linear time.
3. The **HEAPSORT** procedure: sorts an array in place.
   - Runs in *O(n log n).*

Heap - Max Heapify

Maintaining the heap property:

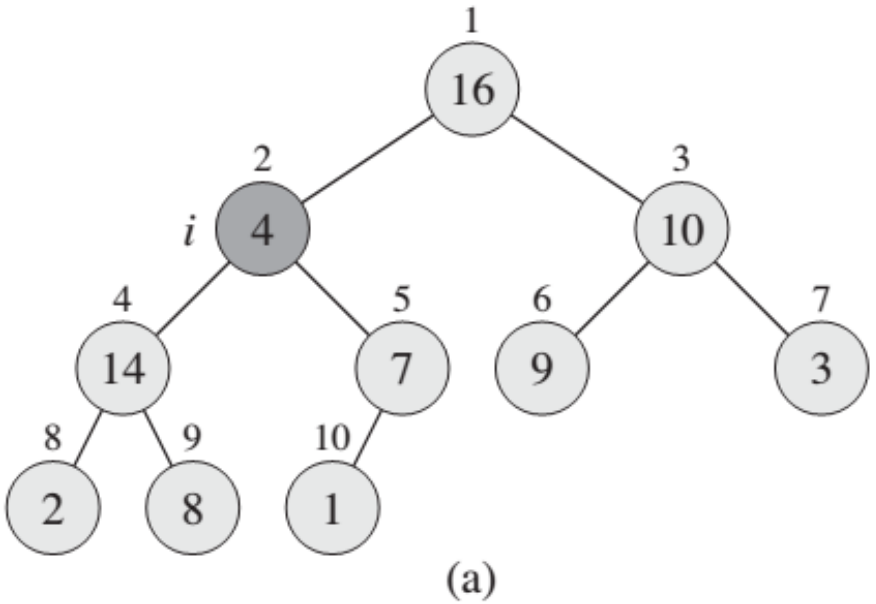
- In order to maintain the max-heap property, we call the procedure **MAX-HEAPIFY**.
- Its inputs are an array **A** and an index **i** into the array.
- When it is called, **MAX-HEAPIFY** assumes that the binary trees rooted at **LEFT(i)** and **RIGHT(i)** are max- heaps, but that **A(i)** might be smaller than its children.

Heap - Max Heapify

```
MAX-HEAPIFY(A, i)
 1  l = LEFT(i)
 2  r = RIGHT(i)
 3  if l ≤ A.heap-size and A[l] > A[i]
 4      largest = l
 5  else largest = i
 6  if r ≤ A.heap-size and A[r] > A[largest]
 7      largest = r
 8  if largest ≠ i
 9      exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

Heap - Max Heapify

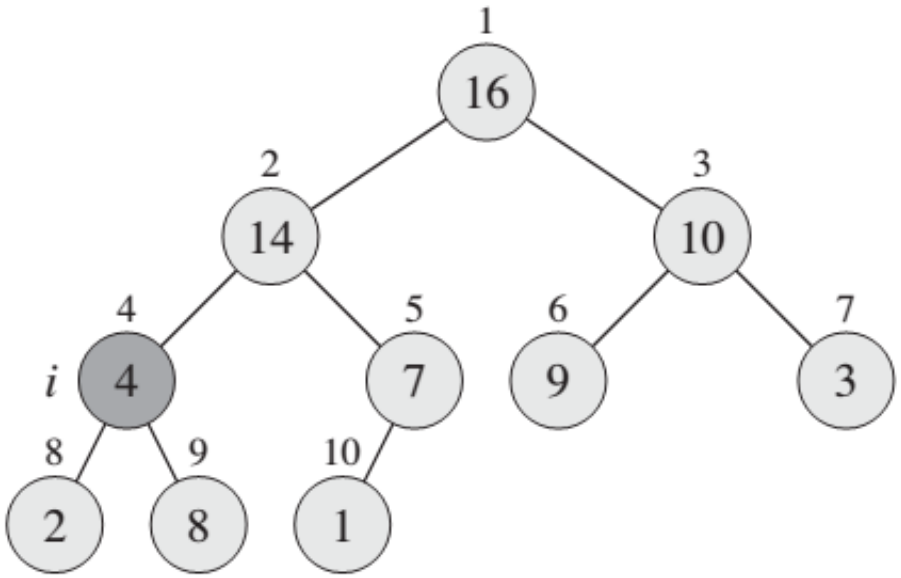
Heapify(A, 2)




```
MAX-HEAPIFY(A, i)
1   l = LEFT(i)
2   r = RIGHT(i)
3   if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5   else largest = i
6   if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8   if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

1	2	3	4	5	6	7	8	9	10
16	**4**	10	**14**	7	9	3	2	8	1

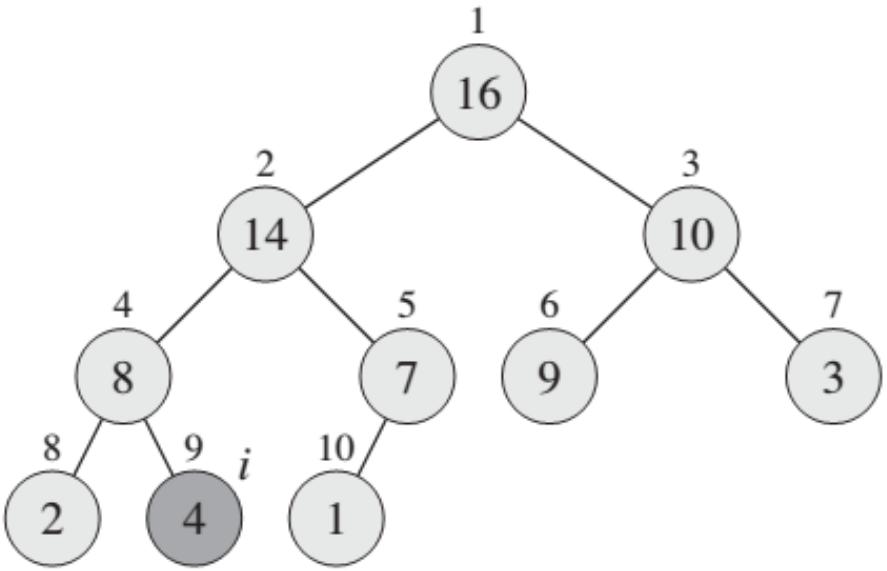
Heap - Max Heapify




```
MAX-HEAPIFY(A, i)
1   l = LEFT(i)
2   r = RIGHT(i)
3   if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5   else largest = i
6   if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8   if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	**8**	1

Heap - Max Heapify

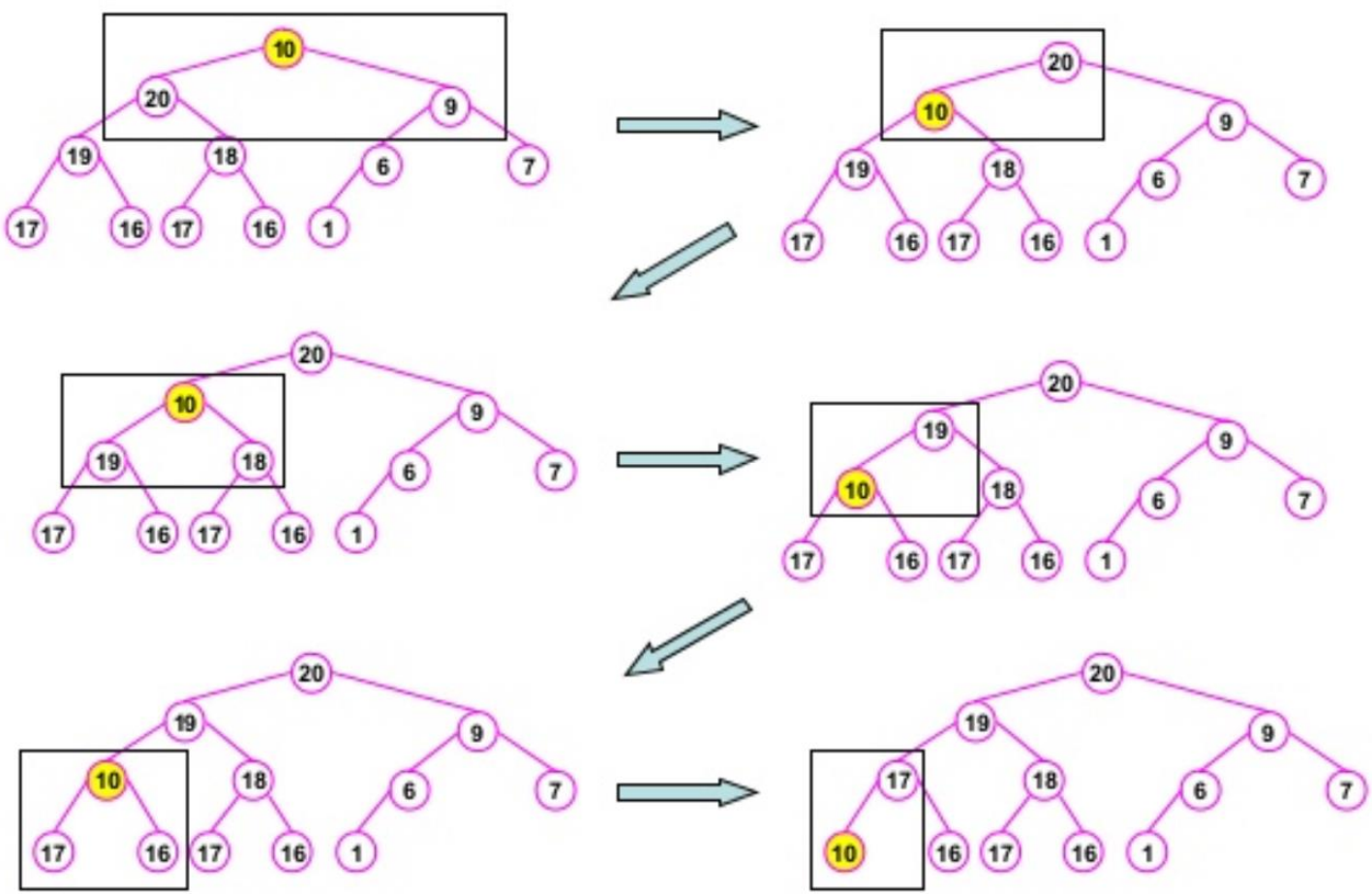



```
MAX-HEAPIFY(A, i)
1   l = LEFT(i)
2   r = RIGHT(i)
3   if l ≤ A.heap-size and A[l] > A[i]
4       largest = l
5   else largest = i
6   if r ≤ A.heap-size and A[r] > A[largest]
7       largest = r
8   if largest ≠ i
9       exchange A[i] with A[largest]
10      MAX-HEAPIFY(A, largest)
```

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**8**	7	9	3	2	**4**	1

Heap - Max Heapify

When call MAX-HEAPIFY(A,1):



Heap - Max Heapify

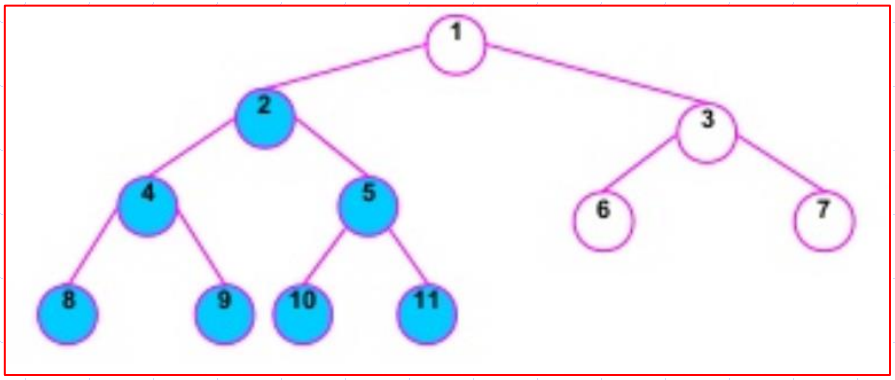
- The running time of MAX-HEAPIFY on a subtree of size n rooted at given node i is:
  - $\Theta(1)$ time to fix up the relationships among the elements A[i], A[l] and A[r],
  - Plus the time to run MaxHeapify() on a subtree rooted at one of the children of node i.
- The children's subtrees each have size at most 2n/3 (why !)

Heap - Max Heapify

Why half-filled case is the worst?

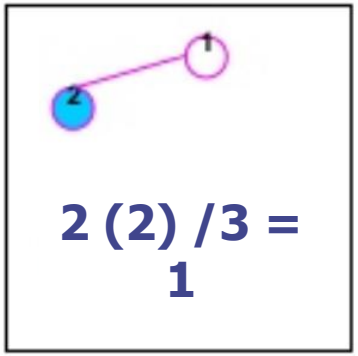
Example:

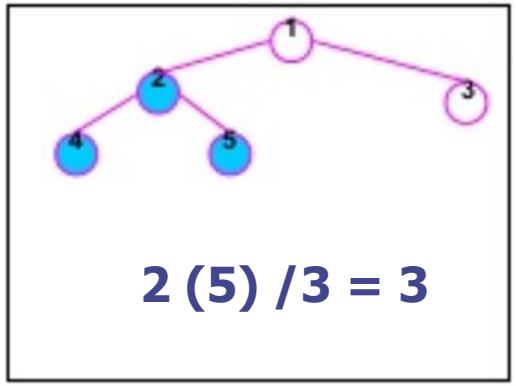
- Tree of height 3.
- Try all possible of sub-trees (different values of **n**) to fill the last level.
- The worst case has the largest value.

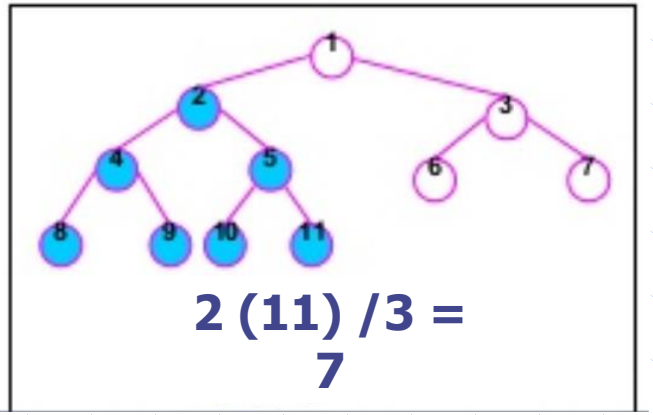


Heap - Max Heapify

- Example:
  - trees of height 1, 2 and 3.
  - The size of left subtree is 2n/3







Heap - Max Heapify

- The time taken by MaxHeapify() is given by

  $T(n) \leq T(2n/3) + \Theta(1)$

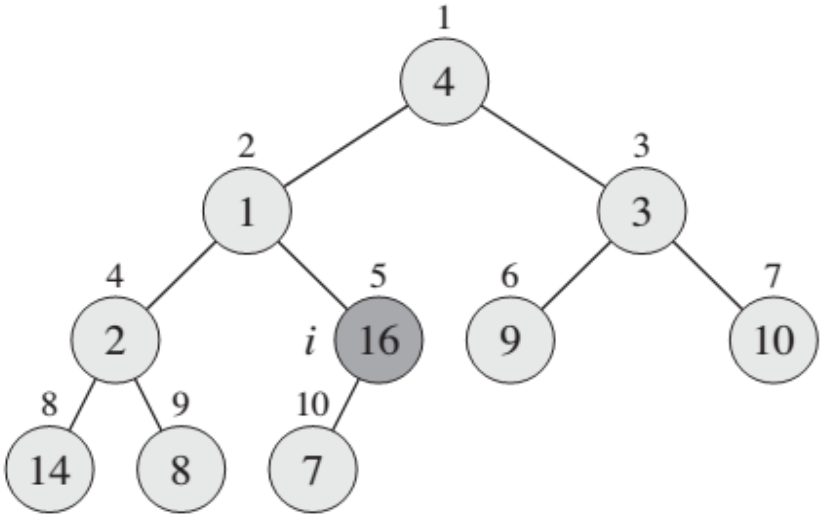
  Solving recurrence using master method (case 2),

  $T(n) = O(\log n)$

- Alternatively, we can characterize the running time of MaxHeapify() on a node of height h as O(h).

Heap – Build Max Heap

- The elements in the subarray $A[(\lfloor n/2 \rfloor + 1) .. n]$ are all **leaves** of the tree, and so each is a 1-element heap to begin with.
- The procedure BUILD-MAX-HEAP goes through the remaining nodes of the tree and runs MAX-HEAPIFY on each one.



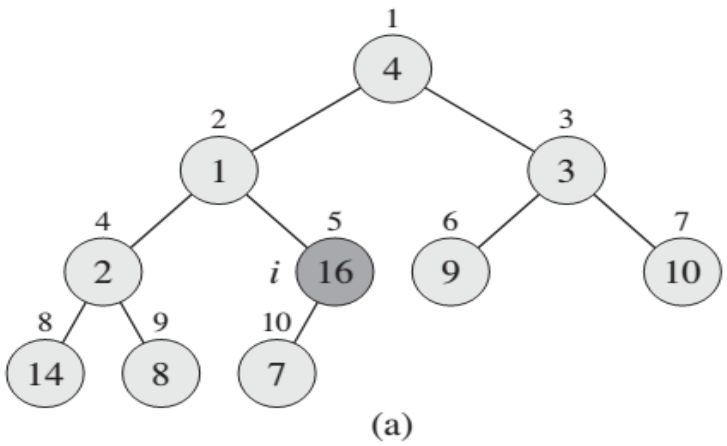
Heap – Build Max Heap

```
BUILD-MAX-HEAP(A)
1  A.heap-size = A.length
2  for i = ⌊A.length/2⌋ downto 1
3      MAX-HEAPIFY(A, i)
```

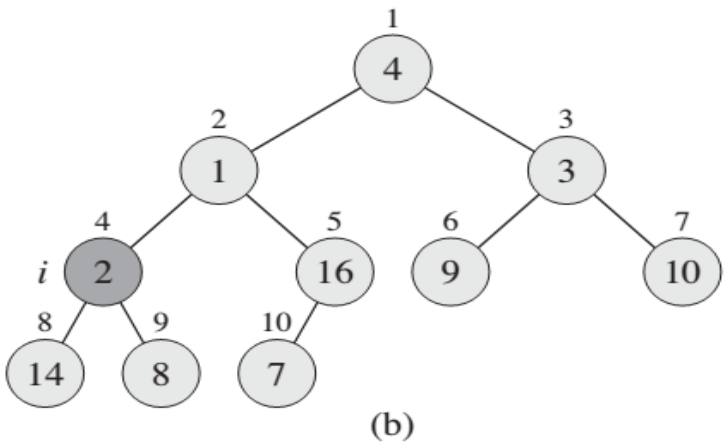
- Upper bound on the running time of BUILD-MAX-HEAP is:
  - Each call to MAX-HEAPIFY costs O(log n) time and
  - BUILD-MAX-HEAP makes O(n) calls.
  - Thus, the running time is O(n log n).
- Upper bound is **not asymptotically tight**.

Heap – Build Max Heap

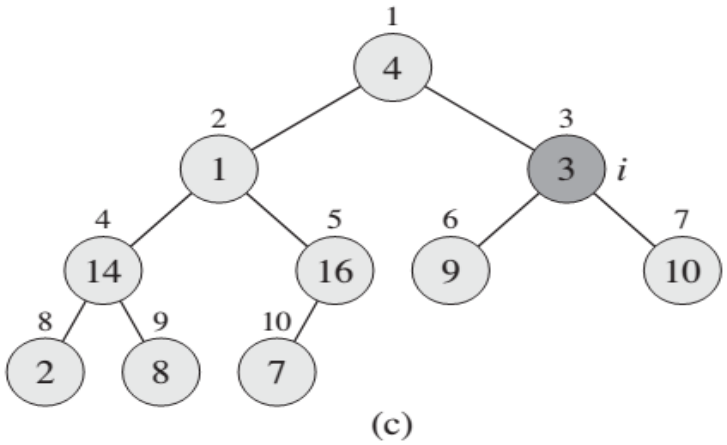
A	4	1	3	2	16	9	10	14	8	7



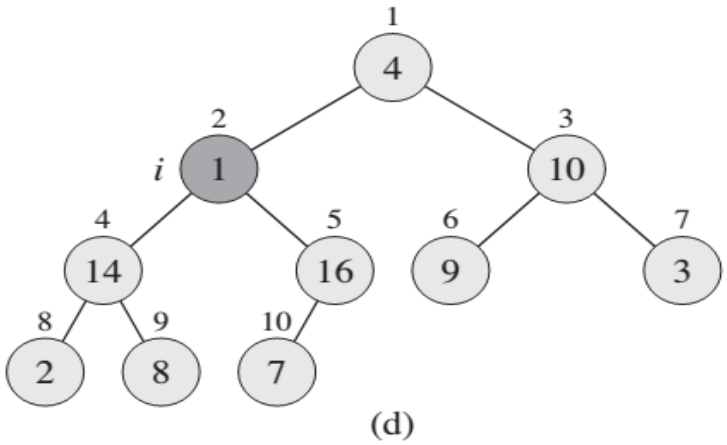
(a)



(b)

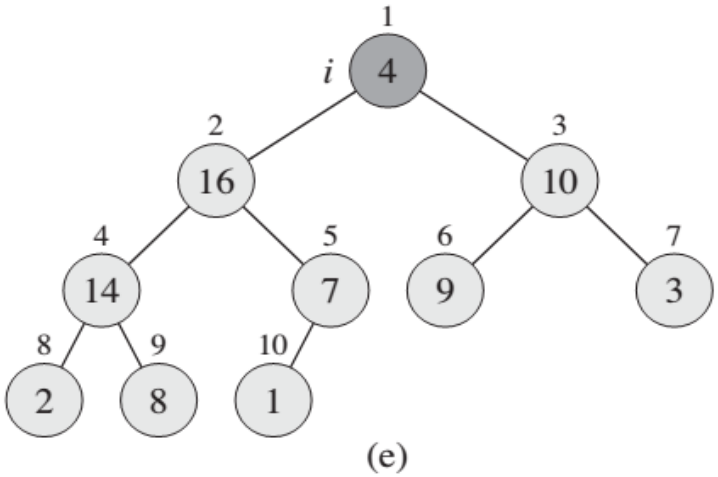


(c)

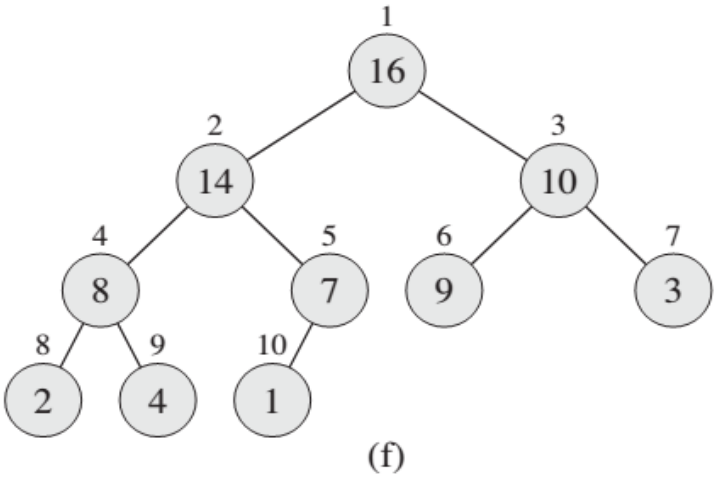


(d)

Heap – Build Max Heap

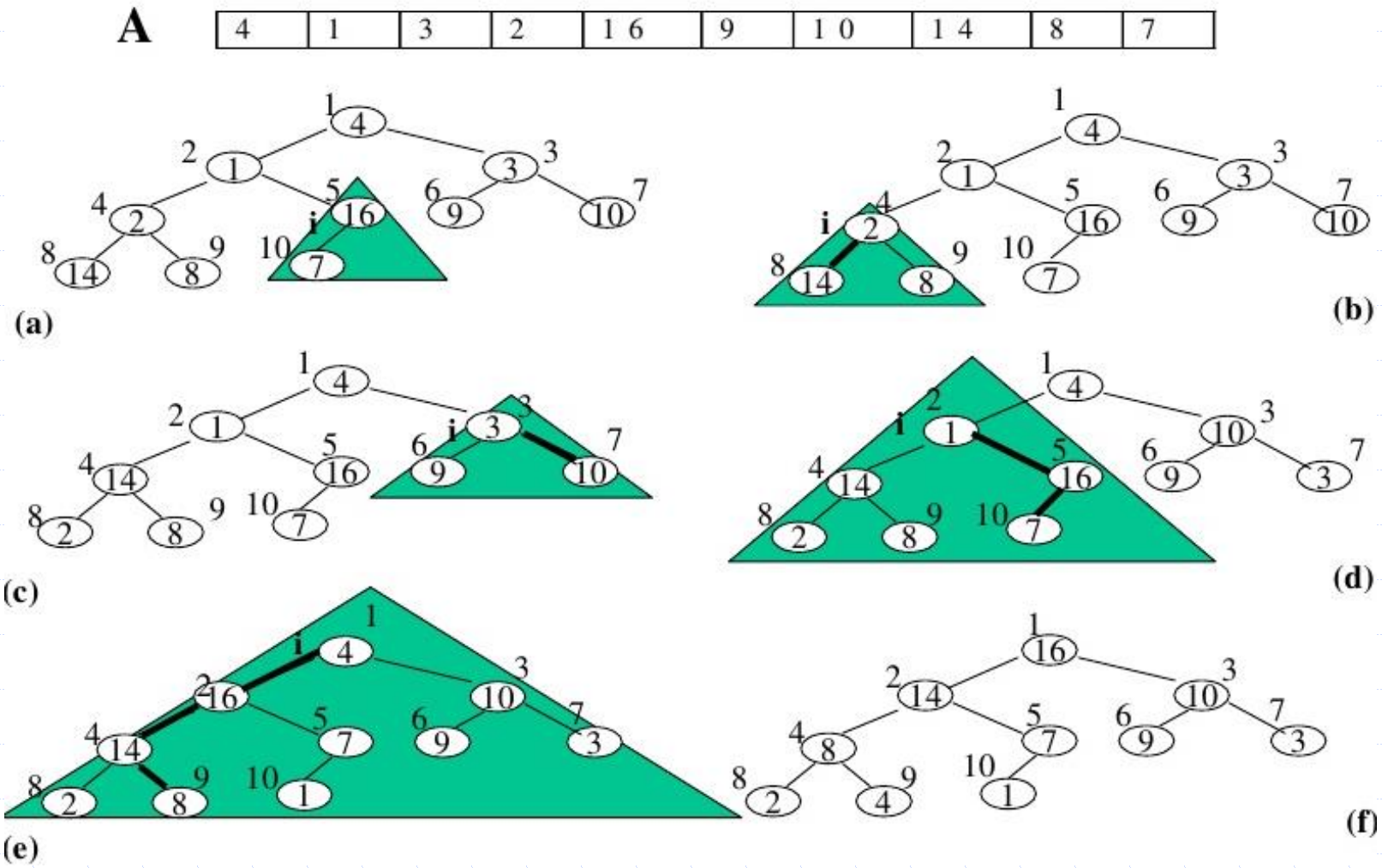


(e)



(f)

Heap – Build Max Heap



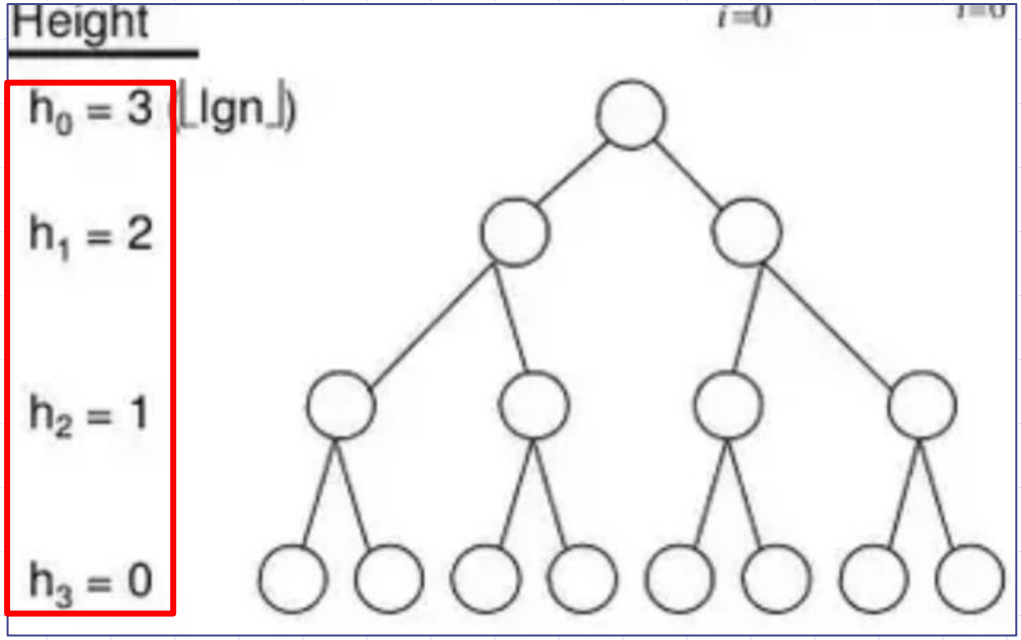
Heap – Build Max Heap

- We can derive a tighter bound by observing that the time for MAX-HEAPIFY to run at a node varies with the height of the node in the tree.
- Our tighter analysis relies on the properties that :
  1. an n-element heap has height $\lfloor \log n \rfloor$ and,
  2. at most $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ nodes of any height h
- The time required by MAX-HEAPIFY when called on a node of height h is O(h), and so we can express the total cost of BUILD-MAX-HEAP as being bounded from above by:

Heap – Build Max Heap

Note: $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ nodes of any height h:

- $\frac{15}{2^{3+1}} = \left\lceil \frac{15}{16} \right\rceil = 1$
- $\frac{15}{2^{2+1}} = \left\lceil \frac{15}{8} \right\rceil = 2$
- $\frac{15}{2^{1+1}} = \left\lceil \frac{15}{4} \right\rceil = 4$
- $\frac{15}{2^{0+1}} = \left\lceil \frac{15}{2} \right\rceil = 8$



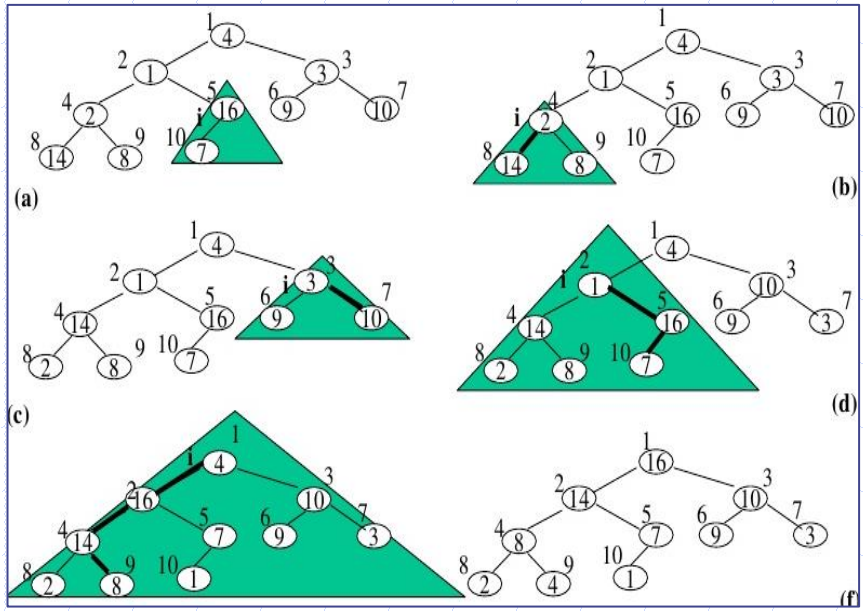
Build-Max-Heap (cont.)

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) .$$

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2 .$$

Using: $$\sum_{k=0}^{\infty} k x^k = \frac{x}{(1-x)^2} \quad \text{for } |x| < 1 .$$

$$O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(n) .$$



Heapsort

- The Heapsort algorithm starts by using BuildMaxHeap() to build a max-heap on the input array A[1..n], where n=length[A].
- The maximum element of the array is stored at the root A[1].
- Swap A[1] with element at A[n] to put the maximum element into its correct position.

Heapsort

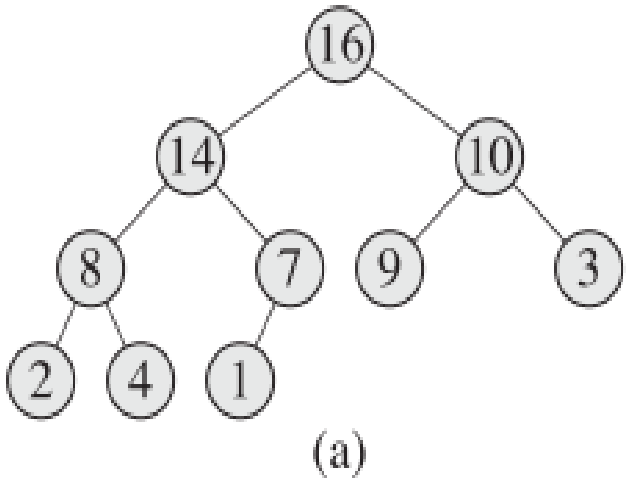
- Then discard the node n from the heap, which will decrement the heap_size[A] by one.
- The children of the root remain max-heap, but the new root element may violate the max-heap property.
- So, restore the max-heap property at A[1] by calling MaxHeapify(A,1) which leaves a max-heap in A[1..(n-1)].
- Repeat the process for the heap_size(A) down to 2.

Heapsort (cont.)

```
HEAPSORT(A)
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```

Heapsort (cont.)

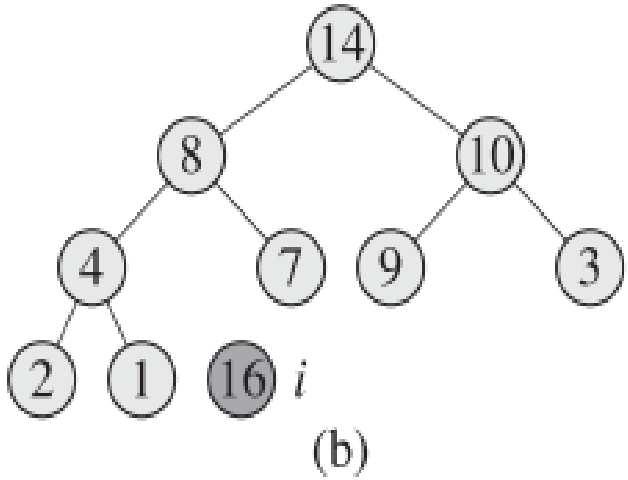
16	14	10	8	7	9	3	2	4	1



(a)

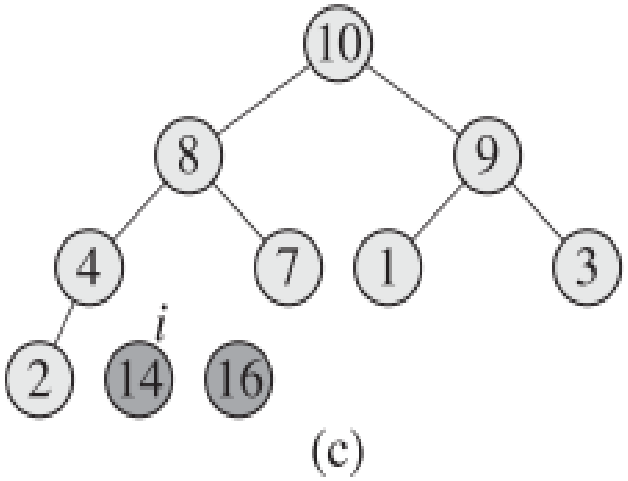
Heapsort (cont.)

14	8	10	4	7	9	3	2	1	16



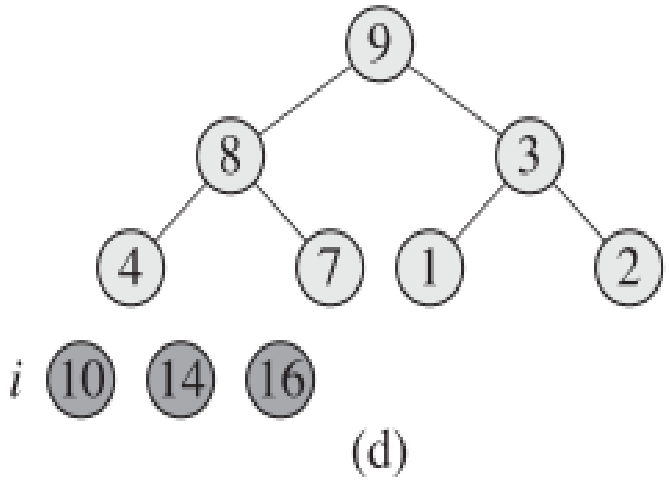
Heapsort (cont.)

10	8	9	4	7	1	3	2	14	16



Heapsort (cont.)

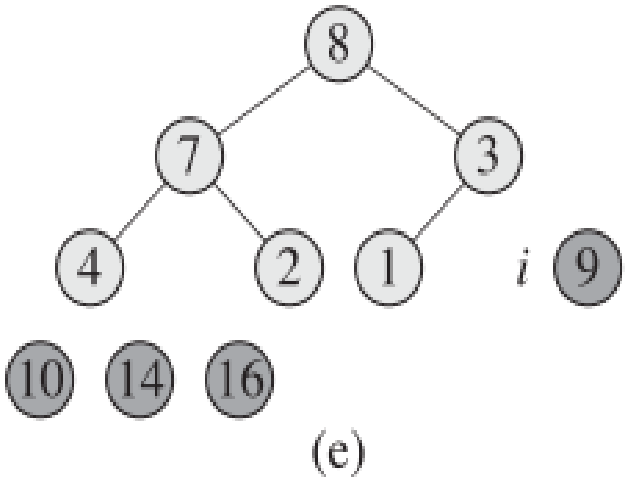
9	8	3	4	7	1	2	10	14	16



(d)

Heapsort (cont.)

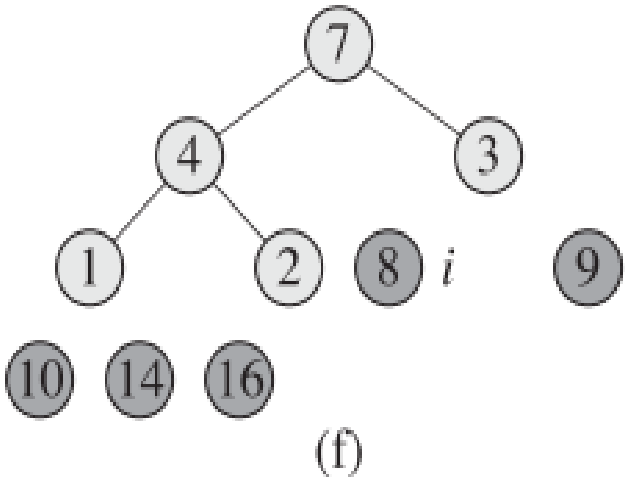
8	7	3	4	2	1	9	10	14	16



(e)

Heapsort (cont.)

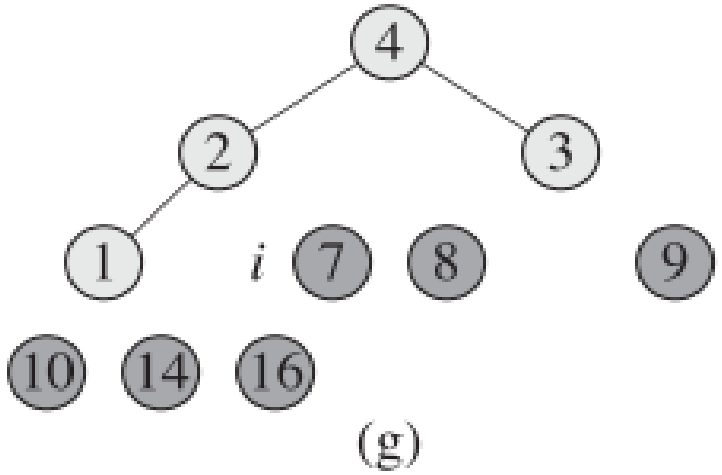
7	4	3	1	2	8	9	10	14	16



(f)

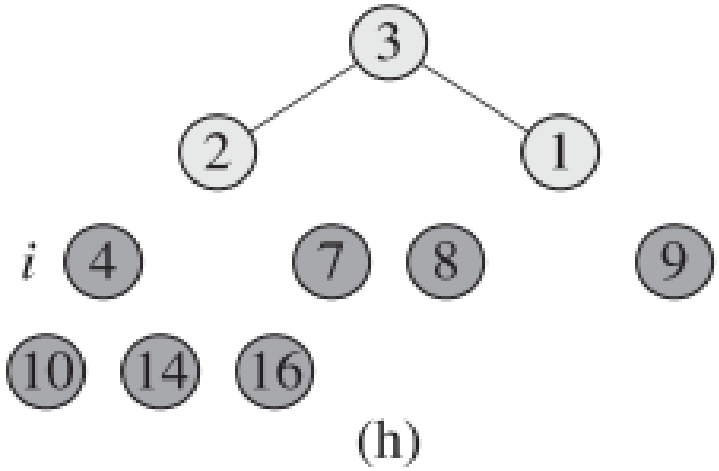
Heapsort (cont.)

4	2	3	1	7	8	9	10	14	16



Heapsort (cont.)

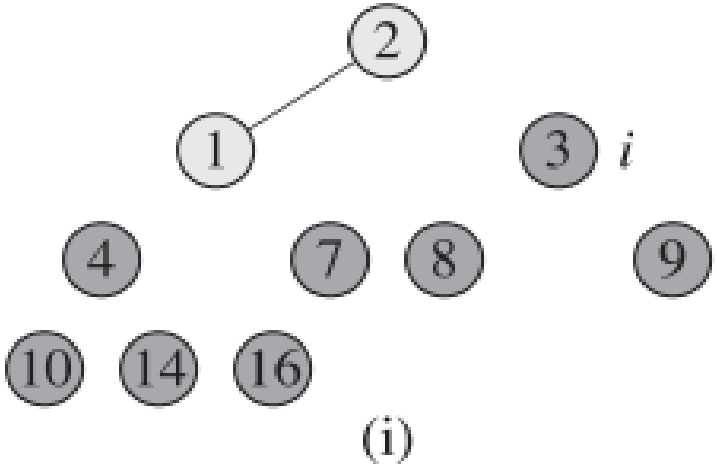
3	2	1	4	7	8	9	10	14	16



(h)

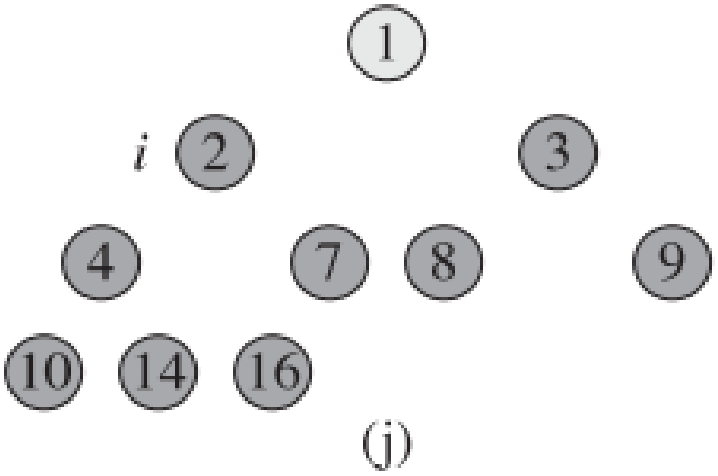
Heapsort (cont.)

2	1	3	4	7	8	9	10	14	16



Heapsort (cont.)

1	2	3	4	7	8	9	10	14	16



Heapsort (cont.)

- The running time of **Heapsort** algorithm is **O(n log n):**
  - **Build-max-heap** takes **O(n)** time
  - (n-1) calls to **max-heapify** which takes **O (log n)**

Heapsort

- Heapsort combines the better attributes of merge sort (speed) and insertion sort (in-place).
  - Running time O(n logn)
  - Sorts in-place